

0.82. Un mur qu'on ne traverse pas : le M

Un mur de M en colonne 10. Avant chaque pas, lire() regarde la case d'arrivée : si c'est un M, on ne bouge pas.

1. Ce qu'il faut comprendre

Le mur est fait de lettres M (`ALPHABET[12]`), posées l'une sous l'autre en colonne 10, des lignes 4 à 12.

Ce qui est nouveau ici : regarder AVANT de bouger. `deplace_croix` ne connaît pas les murs : on refait donc le déplacement à la main. On calcule d'abord la case d'arrivée, `nx` et `ny`. Puis `lire(nx, ny)` (le 0.69) dit ce qu'il y a dessus. Si ce n'est **pas** un M (`!=`), on avance ; sinon, on reste.

`!=` veut dire « n'est pas égal à ». C'est le contraire de `==`.

`chaque(250)` (le 0.77) donne le rythme : au plus un pas tous les 250 ms.

Pour passer, il faut contourner : le mur s'arrête à la ligne 12. En descendant jusqu'à la ligne 13, on peut passer de l'autre côté.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **623 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
main()         227 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), lire(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() — efface des cases
CopierTuiles    40 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage — vide les deux cartes de l'écran
LireManette    37 o ← bouton() — lit les huit boutons
Tuiles         32 o ← CopierTuiles — 2 lettres sur 43
```

Lettres gravées : AM

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,

DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 3	38 o sur place, dans main()	+ AdresseCase (10 o)
bouton()	× 4	20 o sur place, dans main()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
lire()	× 1	11 o sur place, dans main()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES ----

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
```

2. UNE VARIABLE — X

```
uint8_t x = 3;      // la colonne du A : il part de (3, 8), à GAUCHE du mur
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;      // sa ligne
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;      // la colonne d'ARRIVÉE : où le A irait s'il bougeait
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;      // la ligne d'arrivée. On la calcule AVANT de bouger.
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- LE MUR : des lettres M, posées l'une sous l'autre ----
    //
    // colonne :   3 ... 9  10
    // ligne 4 :           M
    // ligne 5 :           M
    // ...           M           le mur : colonne 10, lignes 4 à 12
    // ligne 8 :   A       M           le A part à gauche du mur
    // ...           M
    // ligne 12 :          M
    // ligne 13 :                      ← sous le mur, la ligne 13 est libre : on passe
    //
    // l va de 4 à 12 (<= : « plus petit OU ÉGAL », donc 12 compris).
    for (uint8_t l = 4; l <= 12; l++) {
        poser(10, l, ALPHABET[12]);          // ALPHABET[12] : la 13e lettre, le M
    }

    poser(x, y, ALPHABET[0]);                // le A à sa place de départ

    // ---- LA BOUCLE DU JEU ----
    while (true) {
        image();                             // attend l'image suivante
        if (chaque(250)) {                   // au plus un pas tous les 250 ms (le 0.77)

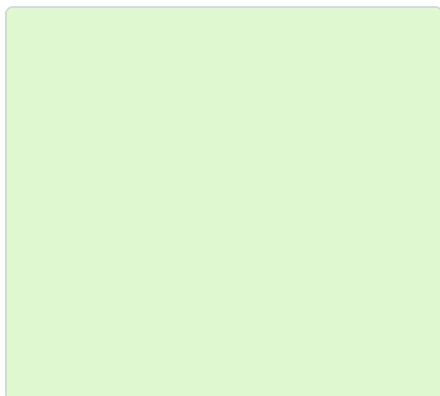
            // ÉTAPE 1 : calculer la case d'ARRIVÉE, sans encore bouger.
            // On part de la case actuelle : si aucune flèche n'est tenue,
            // l'arrivée reste la case où l'on est.
            nx = x;
            ny = y;
            if (bouton(DROITE) && x < 19) { nx = x + 1; } // une colonne à droite
            if (bouton(GAUCHE) && x > 0) { nx = x - 1; } // une colonne à gauche
            if (bouton(BAS) && y < 16) { ny = y + 1; } // une ligne plus bas
            if (bouton(HAUT) && y > 0) { ny = y - 1; } // une ligne plus haut

            // ÉTAPE 2 : REGARDER ce qu'il y a sur la case d'arrivée, avec lire() :
            //
            // lire(nx, ny) != ALPHABET[12]
            // |           | |
            // |           | +-- le M, la lettre des murs
            // |           +----- != : « n'est pas »
            // +----- ce que montre l'écran à la case d'arrivée
            //
            // Exemple : le A est en (9, 8), la flèche droite est tenue.
            // L'arrivée est (10, 8). lire(10, 8) rend le M du mur :
            // « n'est pas un M » est FAUX → le A ne bouge pas.
            if (nx != x || ny != y) {          // une flèche est tenue (|| : OU)
                if (lire(nx, ny) != ALPHABET[12]) {

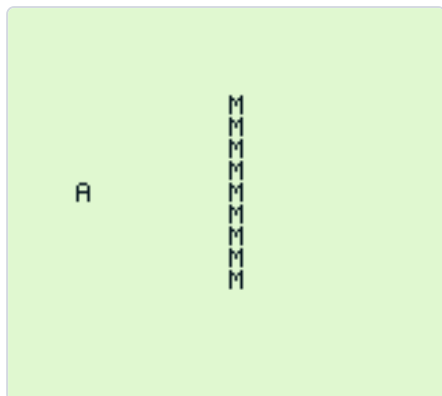
                    // ÉTAPE 3 : ce n'est pas un mur, on avance.
                    effacer(x, y, 1);          // efface le A à l'ancienne case
                    x = nx;                    // la case d'arrivée devient
                    y = ny;                    // la case du A
                    poser(x, y, ALPHABET[0]);  // pose le A à la nouvelle case
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



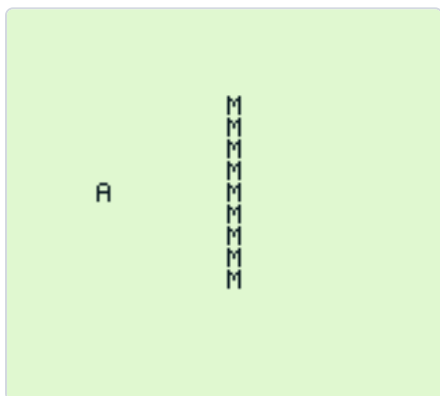
la toute première image — la console vient de commencer — image 1



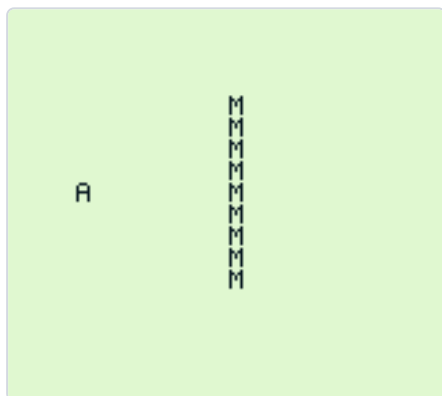
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

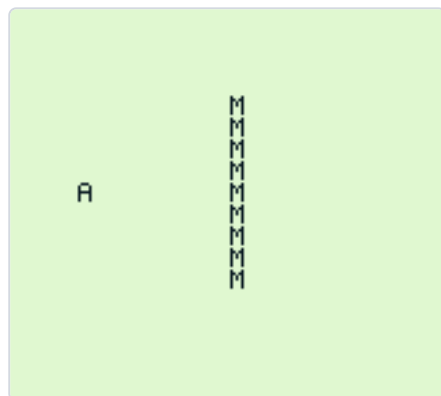
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



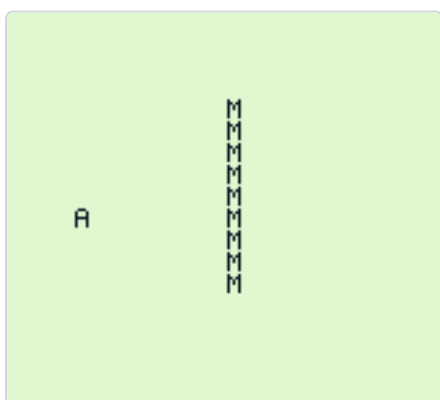
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran n'a pas bougé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un A à gauche d'un mur de M. La flèche droite le bloque contre le mur ; en descendant sous le mur, on passe.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le mur l'arrête en colonne 9 ($x = 9$)

✓ le mur est intact

✓ par-dessous, il passe de l'autre côté ($x = 16$)

Ensuite : **0.82.1. Un mur qu'on ne traverse pas : le M — un labyrinthe** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++